Operating System HW3

B10915061 詹宗叡 四資工三甲

1. 首先，我們先列出 mutual exclusion, progress, and bounded waiting 的 meaning：

- Mutual Exclusion:
  If process Pi is executing in its critical section, then no other processes can be executing in their critical sections
- Progress:
  If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then
  - Only those processes that are not executing in their remainder sections can participate in which will enter their its critical section next
  - This decision cannot be postponed indefinitely.
- Bounded waiting:
  After a process has made a request to enter its critical section and before that request is granted

我們回到下面的 code：

```
1   while (true) {
2     waiting[i] = true;
3     key = 1;
4     while (waiting[i] && key == 1)
5       key = compare_and_swap(&lock,0,1);
6     waiting[i] = false;
7
8      // critical section
9     j = (i + 1) % n;
10    while ((j != i) && !waiting[j])
11      j = (j + 1) % n;
12    if (j == i)
13      lock = 0;
14    else
15      waiting[j] = false;
16    // remainder section
17  }
```

接著我們從 critical section（以下簡稱 cs）的 entry section 得知，進入 cs 的條件為 1. waiting[index] = false / 2. key = 0 ，而只有 lock ＝ 0，key 才會是 0。因此最後我們得到進入 cs 的 condition 為：

1. waiting[index] = false / 2. lock = 0

我們先從 Mutual Exclusion 看起，根據它的定義，我們如果要證明 mutual exclusion 成立，則要證明沒辦法讓兩個（以上）的 process 進入 cs，從上述的 code 來看，line 5 的 key ＝ compare_and_swap(&lock, 0, 1)判定 lock 是不是等於 0，若是則設定 key＝0 和 lock＝1。因此當一個 process 跑完 line5 的 compare_and_swap 後得出結果 lock＝1，如果目前 cpu 進行 context switch，由於 lock＝1，其他的process 仍然無法進入 cs，而以上的結果符合 mutual exclusion 的定義，得出結論 mutual exclusion 成立。

第二個我們要來看 Progress，要符合 progress 有兩個條件：

1. 當沒有 process 要進入 cs 時，不能阻擋想進入 cs 的 process
2. 要在時間內從多個想進入 cs 的 process 挑出一個進去

簡單來說 progress 的意思就是進「想進去」cs 的人「馬上」進去。

回到 code 中，我們到了 exit section 的部分。這一部分的內容是要在多個 process 中以 loop 的模式去找出 waiting[j]為 true 的 process，即找出正在等候的 process，而結果只會有兩種：有找到或沒找到。

- 沒有找到：從 code 的 line12、13 我們得到如果 i==j 時（跑到最後都沒找到），lock＝0，則沒有process 要進入 cs。
    - (Progress Condition 1)
- 找到了：意味著有 process 想進入 cs，從 code 的 line15 可見，他會把 waiting[j]設做 false。記得 entry session 其中一條件為 waiting[index] = false，可見讓這個 process 的條件是改變 waiting[index]，便能夠跑到 compare_and_swap 使 lock＝1，即 cs 上鎖了其他的 process 都不能進去 cs，再加上 line2 可見，除了 process j 以外的 process 的 waiting value 都設為了 true，只有 j 為 false，因此即便有 process j 以外的 process 要進入 cs 也會因為大門已經鎖上而被拒之門外。由此可見，找到等候的 process(j)的情況下，只有 j 能夠進入 cs 裡面。
    - (Progress Condition 2)

從以上的描述可以得知，不管是找到想進去 cs 的 process 還是沒找到 process 要進去 cs，都符合 mutual exclusion 和 progress。

最後的是 bounded waiting，簡單一句話就是即便進入 cs 也不能佔著不出來。我們重新再看一次上面的 code，可以看到他是以 loop 的模式去找出想進去 cs 的 process，也就是一個 process(假設為 j)進去 cs 並完成以後就會出來，然後從這個 j 開始往後再一個一個找出想進去 cs 的 process，因此可以看出，即便這個 process 完成後又想再做一次，也要重新再排過隊，不能一直佔著線。以上的描述也符合 bounded waiting 的要求。

綜合上面幾點，得出 Mutual Exclusion, Progress, Bounded Waiting 都成立

2. (a) Is it possible that many processes receive the same number?

可能，我們把取號碼看成三個動作，挑最大值、加 1、最後把結果 assign 給 number[i]；我們假設目前最大的 value 為 k，且 Process P_i 和 P_j 會出現有相同號碼的情況如下：

1. P_i choosing[i] = true ’context switch 到 P_j choosing[j] = true
2. 接著執行 P_i Max(Number[0], …Number[n-1]) ’然後＋1 ’得出結果 k+1 但還沒 assign 給 Number[i]就 content switch 到 P_j Max(Number[0], …Number[n-1]) ’＋1 ’再 assign 給 Number[j] ’得出 Number[j] = k + 1
3. 最後回到 P_i 讓 k+1 assign 給 Number[i] ’則 Number[i] = k+1
4. 得出結論有可能得到同一個 number ：
   Number[i] = Number[j] = k + 1

(b) If we remove the first while loop (i.e. while(Choosing[j]);), the mutual exclusion is still maintained?

不能，mutual exclusion 無法成立。首先我們要知道刪掉的這個 loop 在做的是讓想要比較的 process 拿號碼牌，接著如上題，我們一樣假設有兩個 process P_i 和 P_j ，而這兩個 process 都想進入 cs，P_i 的 pid<P_j pid，且 choosing[0]-choosing[n-1]的 value 都是 0：

1. P_i choosing[i] = true 後就 context switch 到 P_j choosing[j] = true
2. 接著馬上又 context switch 回 P_i Max(Number[0], …Number[n-1]) ’得出結果 0+1 = 1 ’但還沒 assign 給 Number[i]就又 context switch 回 P_j Max(Number[0], …Number[n-1]) ’＋1 ’再 assign 給 Number[j] ’得出 Number[j] = 1
3. 接著 P_j 繼續執行 ’設定 choosing[j] = false 後開始 loop while((Number[j]!=0)&&((Number[j],j)<(Number[i],i))); 順利執行完成後由於 Number[i]還沒被 assign ’因此 P_j 判斷目前沒有別的 process 會阻擋 ’P_j 順利進入 cs
4. context switch 回 P_i 並完成 assign，Number[i] = 1
5. 接著 P_i 設定 choosing[i] = false
6. 最後 P_i 開始 for 迴圈，由於Number[i]<Number[j]但 Process ID I < j，P_i 也判斷沒有 process 會阻擋 P_i，因此 P_i 進入 cs

從以上六點可見，如果刪去第一個 while loop 就有可能讓兩個 process 一起進入 cs，違反了他的定義，因此 mutual exclusion 不成立。

(c) Please prove the correctness of Bakery algorithm in terms of mutual exclusion, progress, and bounded waiting.

參考第一題的作業，已經知道以上三個的成立條件。

在 Bakery algorithm 中，process 想進入 cs 的 entry section 是：

- 想進入 cs 的 process(index)都拿好號碼牌（choosing[index] = false）
- 想進入 cs 且拿好號碼牌的 process(Number[index] != 0)中，number 都比自己大，或 number 相同但 process id 比自己大

我們假設 mutual exclusion 不成立，而現在有兩個 process P_i 和 P_j 要同時進去 cs，我們再假設 Number[i] < Number[j]且 process ID i<j，意思就是 P_i 先進入 cs，P_j 要進入 cs，有兩個可能性：choosing[i] == false 或 Number[j] < Number[i]。而 Number[j] < Number[i]是不可能成立的，因為 P_i 是先進入 cs 的，則只有 choosing[i] == false P_j 才能進入 cs，但是 choosing[i] == false 代表 P_i 還沒領到號碼牌，而 P_j 已經領到了，這樣的情況下 P_i 不可能比 P_j 先進入 cs，因此假設與實際情況相反，得出結論mutual exclusion 成立。

接著就是 progress，

- 只有一個 process 想進去 cs：我們先假設只有 process P_i 想進入 cs，則：除了 Number[i]以外的 Number 都為 0，且由於他們的choosing 都是 false，因此 P_i 可以順利通過第一個 while loop，而又由於只有 number[i] != 0，沒有 number 會比他更小了，所以第二個 while loop 也可以順利進行。由此可見，當只有一個 process 想進入 cs 時，不會有任何阻擋，能順利進去。
- 有多個 process 想進去 cs：由 bakery algorithm 可見，只要 process 抽好號碼牌 choosing 就會變成 false（即抽完），所有第一個 while loop 只會在抽的時候停住，不會一直停住。接著我們可以知道，一定會有一個 process 的 number 最小，即便有一樣 number 的也可以按照 process

id 來決定，而這一部分的判斷是由第二個 while loop 來決定，如果有相同 number 的兩個 process 想進入 cs，第二個 while loop 會比較他們的 number 和 process id，較小的 process id 或 number 能夠進入 cs，較大的就會跳出 while loop。因此即便有多個 process 想進去 cs，也能夠在時間內挑出最適合的 process 進入 cs。

由以上兩個情況都符合 progress 的條件，因此 progress 也成立

最後便是 Bounded Waiting，我們從他的 exit section 可以知道每當一個 process 執行完 cs，就會 initialize 他的 number 為 0，這樣當他跑到第二個 while loop 中的 Number[j] != 0 這個條件時，會因為 number=0 而跳出這個 while loop，因此 process 並不會無限期的一直等待，會有一段時間的限制，因此 bounded waiting 也成立。

從以上的說明中，證明 mutual exclusion、progress 和 bounded waiting 都成立。

3.

```
// acquire(): get lock, release(): release lock
void acquire() {
        while(compare_and_swap(&lock, 1, 0) == 0);
}
void release() {
        lock = 1;ss
}
```

4.

Producer

```
do { // produce an
     item in nextp
   wait (mutex);
   wait (empty);
   // add the item to
      the buffer
   signal (full);
   signal (mutex);
} while (TRUE);
```

Comusmer

```
do {
   wait (full);
   wait (mutex);
   // remove an item from buffer
   to nextc

   signal (mutex);
   signal (empty);
   // consume the item in nextc
   } while (TRUE);
```

首先，我們要知道每一個 Semaphore 的作用：

- mutex： 用來確保每一次只會有一個 process 或 thread 可以修改 buffer 的內容來避免 racing
- full 和 empty：解決 deadlock

接著我們可以看到 wait(mutex); wait(empty);和 signal(full); signal(mutex);，mutex 是在 full 和 empty 的外面，有可能會多過一個 thread 被執行，當目前為全空或全滿時，就會造成 deadlock。

解決方法：每一次增加一個 item 進入 buffer 就 itemcount++，加入 while loop 去判斷 itemcount 是不是跟 buffer 的大小，如果是的話就執行 wait(full)，並加入 if statement if(itemcount==1)就 signal（empty）。

wait(empty); wait(mutex);和 signal(mutex); signal(full); <<改成這樣也是解決方法之一